

Bloque 3 - Tema 4 ejercicios

LENGUAJES DE INTERROGACIÓN DE BASES DE
DATOS. ESTÁNDAR ANSI SQL. PROCEDIMIENTOS
ALMACENADOS. EVENTOS Y DISPARADORES



SOLUCIONES

PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

Enunciado 1

Dado el siguiente esquema de bases de datos:

- provincias(**codpro**, nombre)
- pueblos(**codpue**, nombre, **codpro**)
- **clientes**(**codcli**, nombre, direccion, codpostal, codpue)
- vendedores(**codven**, nombre, direccion, codpostal, codpue, **codjefe**)
- articulos(**codart**, descrip, **precio**, stock, stock_min)
- **facturas**(**codfac**, fecha, **codcli**, codven, iva, dto)
- **lineas_fac**(**codfac**, **linea**, cant, codart, **precio**, dto)

Escriba las sentencias que den respuesta a las consultas solicitadas:

- a) Mostrar el nombre y el código de las provincias.

```
SELECT nombre, codpro FROM provincias;
```

- b) Mostrar el código de factura, número de línea e importe de cada línea.

```
SELECT codfac, linea, cant*precio*(1-dto/100) FROM lineas_fac;
```

- c) Mostrar los distintos tipos de descuentos aplicados por los vendedores cuyos códigos no superan el valor 50.

```
SELECT distinct dto FROM facturas WHERE codven <=50;
```

- d) Mostrar el código y fecha de las facturas del cliente 100 con IVA 16 o con descuento 20%.

```
SELECT codfac, fecha FROM facturas WHERE codcli= 100 and (iva=16 or dto=20);
```

- e) Mostrar todos los artículos cuyo stock no se halla entre el stock mínimo menos 500 unidades y el stock máximo más 500 unidades.

```
SELECT * FROM articulos WHERE stock not between stock_min-500 and stock_min+500;
```

- f) Mostrar el código y nombre de los pueblos pertenecientes a la Comunidad Valencia (Alicante tiene el código de provincia 03, Castellón el 12 y Valencia el 46).

```
SELECT codpue, nombre FROM pueblos WHERE codpro in('03', '12', '46');
```

- g) Mostrar los códigos y fechas de aquellas facturas sin código de cliente o sin código de vendedor.

```
SELECT codfac, fecha FROM facturas WHERE codcli is null or codven is null;
```

- h) Mostrar el precio más caro y el más barato de todos los artículos.

```
SELECT max(precio), min(precio) FROM articulos;
```

- i) Calcular el precio medio de aquellos artículos cuyo stock no supera las 30 unidades.

```
SELECT avg(precio) FROM articulos WHERE stock<=30;
```

- j) Calcular el número de artículos con precio no nulo.

```
SELECT count(precio) FROM articulos;
```

- k) Mostrar el número de precios distintos de los artículos.

```
SELECT count(distinct precio) FROM articulos;
```

- l) Calcular el número de artículos cuyo stock es nulo.

```
SELECT count(*) FROM articulos WHERE stock is null;
```

- m) Mostrar el número de pueblos para cada provincia.

```
SELECT codpro, count(*) FROM pueblos GROUP BY codpro;
```

- n) Mostrar el menor código de pueblo para cada provincia.

```
SELECT codpro, min(codpue) FROM pueblos GROUP BY codpro;
```

- o) Obtener el número de facturas para cada cliente y tipo de IVA distinto.

```
SELECT codcli, iva, count(*) FROM facturas GROUP BY codcli, iva;
```

- p) Calcular la media del descuento de las facturas para cada cliente.

```
SELECT avg(dto) FROM facturas GROUP BY codcli;
```

- q) Obtener el número máximo de unidades vendidas de un artículo.

```
SELECT max(sum(cant)) FROM lineas_fac GROUP BY codart;
```

- r) Obtener el código de los pueblos en los que tienen dos o más clientes.

```
SELECT codpue FROM clientes GROUP BY codpue HAVING count(*)>=2;
```

- s) Obtener el código de aquellos artículos que siempre han vendido al mismo precio (Ayuda: un artículo se habrá vendido siempre al mismo precio si en las líneas de factura el precio máximo y el mínimo coinciden).

```
SELECT l.codart FROM lineas_fac l GROUP BY l.codart HAVING max(l.precio)=min(l.precio);
```

- t) Mostrar el código de factura, la fecha y el nombre del cliente destinatario de la factura.

```
SELECT f.codfac, f.fecha, c.nombre FROM facturas f JOIN clientes c USING(codcli);
```

- u) Mostrar el nombre de cada pueblo y el de su provincia.

```
SELECT p.nombre, pr.nombre FROM pueblos p JOIN provincias pr ON p.codpro=pr.codigo;
```

- v) Obtener el nombre de aquellos clientes y vendedores cuyo nombre coincide.

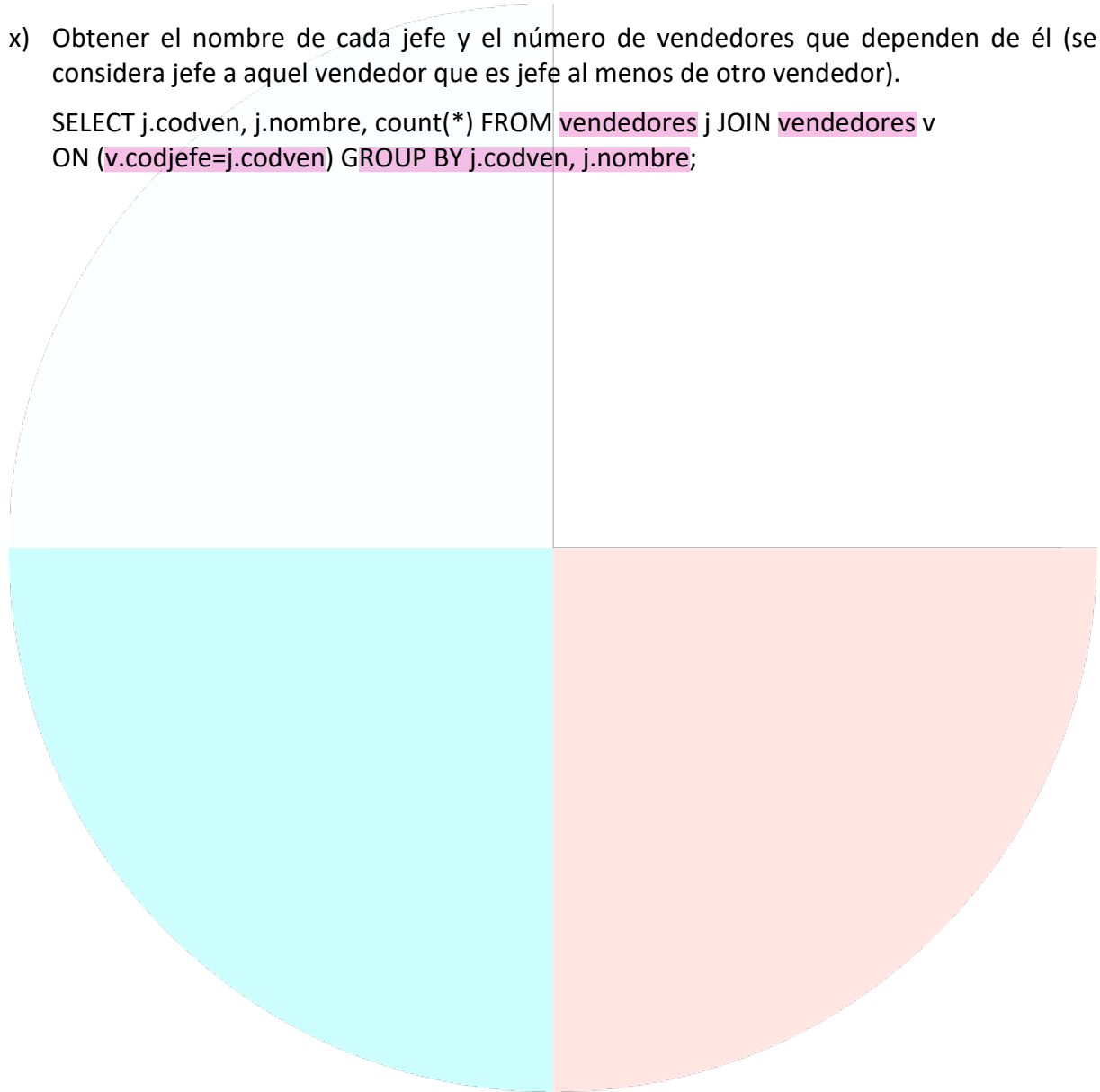
```
SELECT nombre FROM clientes c JOIN vendedores v USING(nombre);
```

- w) Mostrar el código y nombre de cada vendedor, junto al número de facturas que ha realizado.

```
SELECT codven, v.nombre, count(*) FROM vendedores v JOIN facturas f USING(codven)  
GROUP BY codven, v.nombre;
```

- x) Obtener el nombre de cada jefe y el número de vendedores que dependen de él (se considera jefe a aquel vendedor que es jefe al menos de otro vendedor).

```
SELECT j.codven, j.nombre, count(*) FROM vendedores j JOIN vendedores v  
ON (v.codjefe=j.codven) GROUP BY j.codven, j.nombre;
```



Enunciado 2

Teniendo en cuenta el esquema de base de datos del ejercicio anterior, responda a las siguientes cuestiones:

- a) ¿Cuál es la diferencia entre las siguientes dos sentencias?

```
SELECT COUNT(CODART) FROM ARTICULOS;  
SELECT COUNT(*) FROM ARTICULOS;
```

Como el campo codart no puede ser nulo porque es la clave primaria, ambas consultas devuelven el mismo resultado

- b) ¿Qué calcula la siguiente sentencia?

```
SELECT PRECIO FROM ARTICULOS WHERE MAX(PRECIO)=3;
```

Error sintáctico. No se pueden utilizar funciones de agregación en la cláusula WHERE.

- c) ¿Qué realiza la siguiente sentencia?

```
SELECT CODCLI, AVG(DTO), IVA FROM FACTURAS GROUP BY CODCLI;
```

Podemos pensar que la consulta agrupa las facturas por codcli y para cada grupo muestra el código de cliente, el iva y la media de descuento. Sin embargo, en el select no pueden haber campos que no estén definidos en el group by, salvo las funciones de agregación.

- d) ¿Qué realiza la siguiente sentencia?

```
SELECT CODCLI, AVG(DTO)  
FROM FACTURAS  
WHERE CODFAC>100  
GROUP BY CODCLI  
HAVING AVG(DTO)>4;
```

Para las facturas con código de factura superior a 100 las agrupa por codcli y se filtran los grupos cuyo descuento medio es superior a 4. Y para cada grupo resultante muestra el código de cliente y la media de descuento.

Enunciado 3

Responda a las cuestiones que se le plantean:

- a) Disponemos de la tabla COCHES_VENTA. Significado de la sentencia:

```
SELECT modelo, color, COUNT(bastidor) AS num
FROM COCHES_VENTA
GROUP BY modelo, color
HAVING COUNT(bastidor) <= 1;
```

- b) El usuario **museo_consulta** necesita permisos de **selección** en la tabla **ENTIDADES_EXTERNAS**. Escriba la sentencia SQL que satisface los requisitos planteados.

- c) Otorgue permisos al usuario **USUARIO1** para actualizar la columna **SALARIO** de tabla **EMPLEADOS** sin permitirle dar de alta nuevos empleados.

- d) Significado de la siguiente sentencia:

```
SELECT *
FROM Empleados
ORDER BY Provincia DESC, Municipio;
```

- e) Disponemos de las tablas **departamentos** e **investigadores** y se solicita encontrar el **nombre** y **salario** de los investigadores cuyo salario es mayor que el **salario medio de todos los departamentos**. ¿Cómo se obtiene el **tercer salario mínimo de los investigadores de un departamento concreto**?

```
SELECT nombre, salario
FROM investigadores
WHERE salario > ALL (SELECT avg(salario) FROM investigadores GROUP BY dep_id);
```

```
SELECT DISTINCT e1.salario, e1.dep_id FROM investigadores e1, departamentos d1
WHERE e1.dep_id=d1.dep_id AND d1.dep_nombre="TIC" AND 3 =
(SELECT COUNT(DISTINCT salario) FROM investigadores e2, departamentos d2 WHERE
e2.dep_id=d2.dep_id AND d2.dep_nombre="TIC" AND e2.salario<=e1.salario);
```

- f) En una aplicación de gestión de compras de coches se tienen 3 tablas. La tabla **PERSONA** (**IDPERSONA**, **APELLIDO1**, **APELLIDO2**, **NOMBRE**, **MUNICIPIO**), la tabla **COCHE** (**IDCOCHE**, **MARCA**) y la tabla **COMPRAS** (**IDPERSONA**, **IDCOCHE**). Una persona puede comprar varios coches, pero un coche sólo puede ser comprado por una única persona. Indique la consulta que nos devuelve el número de coches comprado de cada marca en cada municipio (registros de la forma 3, HONDA, MADRID)

```
SELECT count(*), c.marca, b.municipio FROM compras a, persona b, coche c
WHERE b.idpersona=a.idpersona and c.idcoche=a.idcoche
GROUP BY c.marca, b.municipio
```

- g) Indique el significado de la consulta y posteriormente especifique el resultado de ejecutar la sentencia SQL:

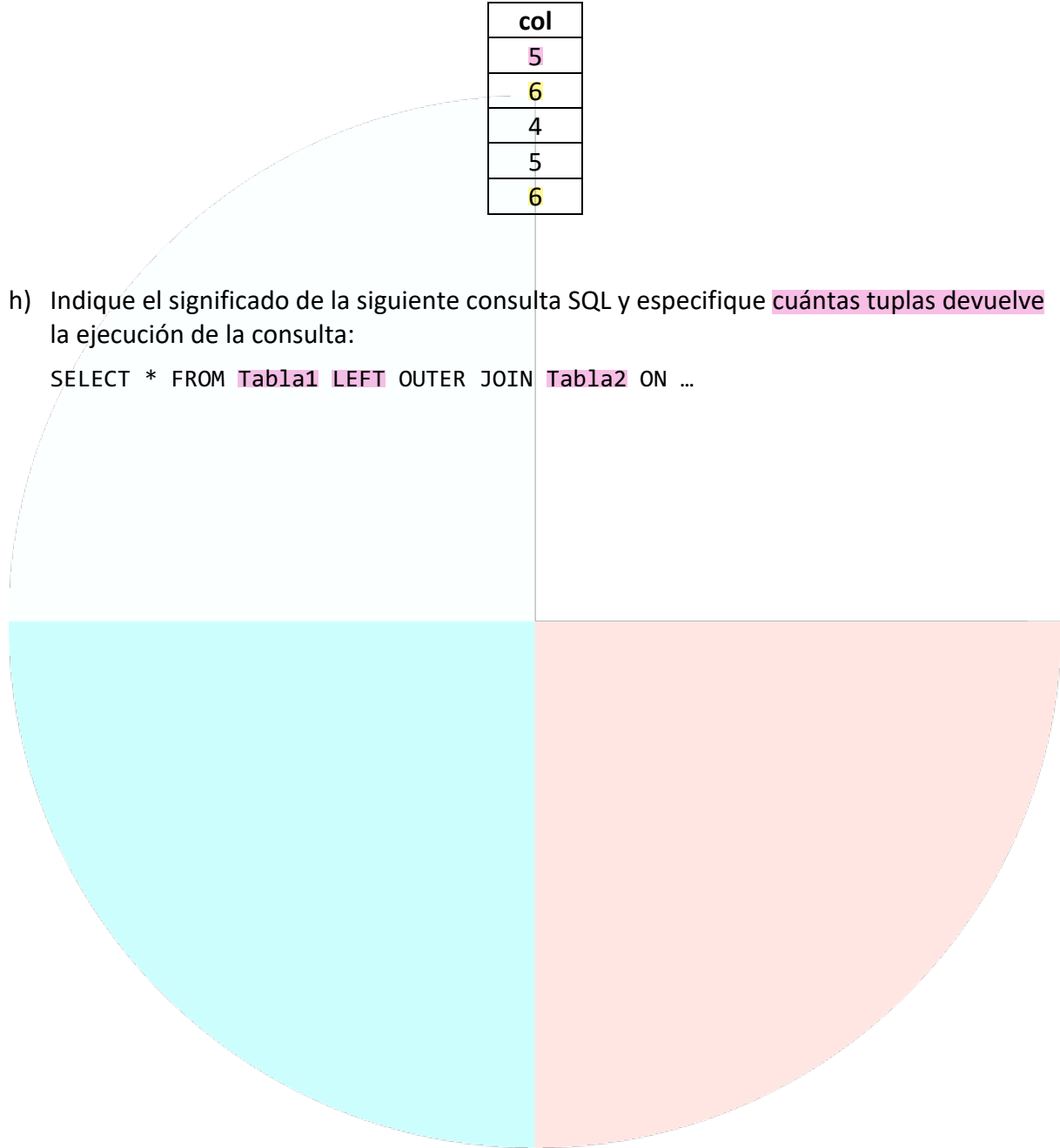
```
SELECT MAX(col) FROM table WHERE col < ( SELECT MAX(col) FROM table );
```

Sobre la siguiente tabla (Tabla1):

col
5
6
4
5
6

- h) Indique el significado de la siguiente consulta SQL y especifique cuántas tuplas devuelve la ejecución de la consulta:

```
SELECT * FROM Tabla1 LEFT OUTER JOIN Tabla2 ON ...
```



Enunciado 4

Dado el siguiente esquema de bases de datos:

- Proveedores, S(**S#**, **nomprov**, **domicilio**, **ciudad**)
- Piezas, P(**P#**, **nompiez**, **color**)
- Proyectos, J(**J#**, **nomproy**, **duración**, **ciudad**)
- Envíos o Suministros, que relaciona a los proveedores que suministran ciertas cantidades de piezas a los proyectos, **SPJ(S#, P#, J#, cantidad)**

Responda a las siguientes cuestiones:

- a) Obtener las p# de las piezas suministradas por un proveedor de "Londres" a un proyecto de "Londres".

```
SELECT DISTINCT p# FROM spj, s, j
WHERE spj.s#=s.s# AND spj.j#=j.j# AND s.ciudad="Londres" AND j.ciudad=s.ciudad;
```

- b) Obtener las p# de las piezas suministradas por un proyecto de "Londres" a un proyecto que no sea de "Londres".

```
SELECT DISTINCT p# FROM spj, s, j
WHERE spj.s#=s.s# AND spj.j#=j.j# AND s.ciudad="Londres" AND j.ciudad!=s.ciudad;
```

- c) Indique el resultado de la siguiente consulta:

```
SELECT s#, p#, j#
FROM spj
WHERE s.ciudad=p.ciudad
AND p.ciudad<>j.ciudad;
```

La sentencia es sintácticamente incorrecta (<>).

- d) Indique el resultado de la ejecución de la siguiente sentencia:

```
UPDATE SPJ
SET CANTIDAD=100
WHERE "Londres" IN(SELECT CIUDAD
FROM S
WHERE S.S#=SPJ.S#);
```

Actualiza a 100 la cantidad de los suministros de todos los proveedores de Londres.

e) Indique el resultado de la ejecución de la siguiente sentencia:

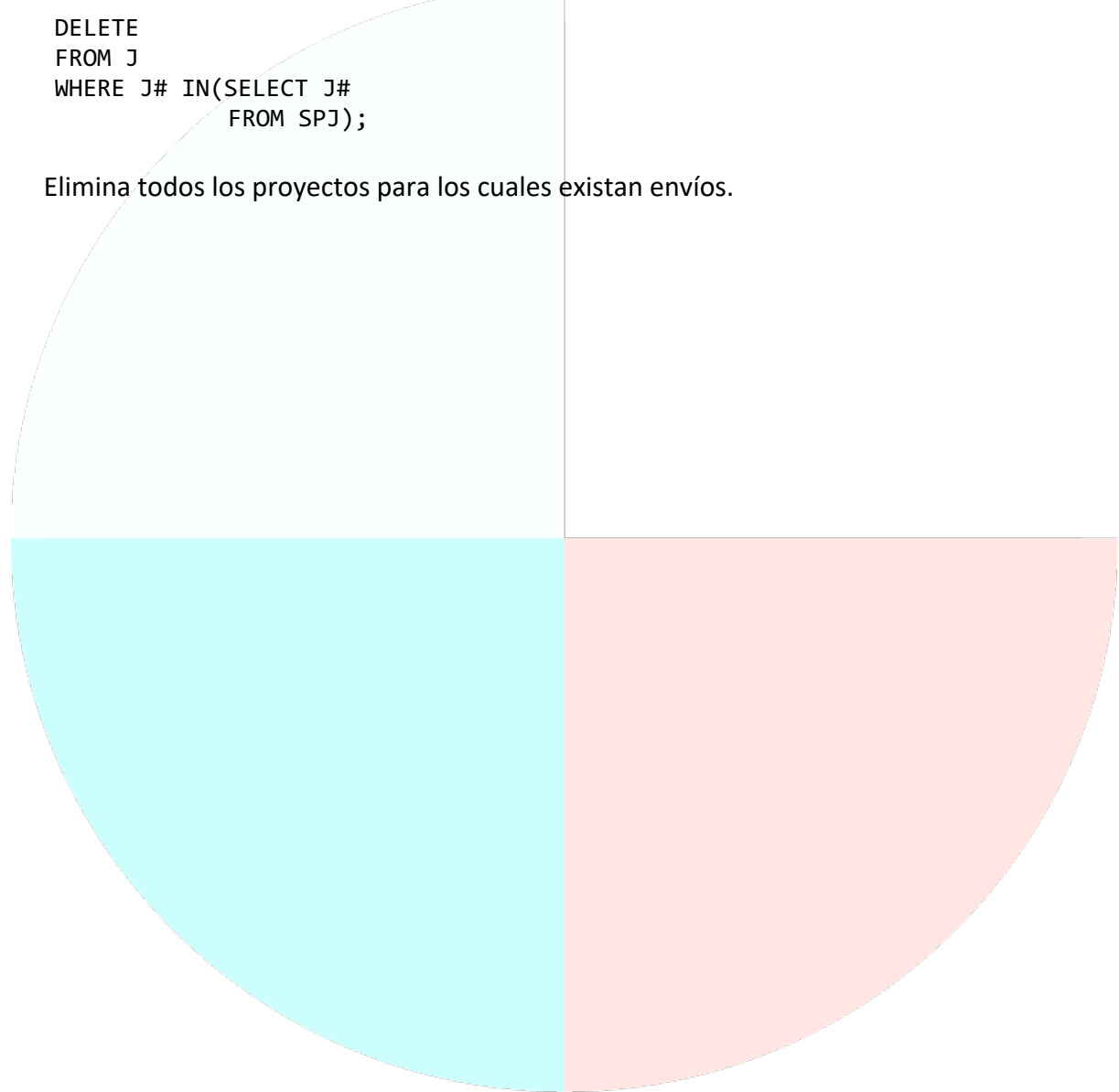
```
UPDATE P
SET COLOR="gris"
WHERE P# IN (SELECT P#
            FROM SPJ);
```

Actualiza a gris el color de todas las piezas que son suministradas.

f) Indique el resultado de la ejecución de la siguiente sentencia:

```
DELETE
FROM J
WHERE J# IN(SELECT J#
            FROM SPJ);
```

Elimina todos los proyectos para los cuales existan envíos.



Enunciado 5

Dado el siguiente esquema de bases de datos:

- **Clases**(clase, tipo, país, numArmas, calibre, desplazamiento)
- **Barcos**(nombre, clase, botado)
- **Batallas**(nombre, fecha)
- **Resultados**(barco, batalla, resultado) donde barco clave ajena con Barcos y batalla clave ajena con Batallas.

Se facilita el fichero **B3T4Ejercicios Batallas.sql** para la creación del esquema y la carga inicial de datos. Escriba las sentencias que den respuesta a las consultas solicitadas teniendo en cuenta que para cada consulta se muestra el resultado que se debería obtener:

- a) Nombres de clases de barco y países de las clases de barco que llevaban armas con un calibre mayor o igual a 16 pulgadas.

clase	país
Colorado	USA
Iowa	USA
North Carolina	USA
Yamato	Japan

```
SELECT clase, país FROM Clases WHERE numArmas>0 AND calibre>=16;
```

- b) Nombre, desplazamiento y número de armas de todos los barcos que participaron en la batalla de Guadalcanal.

clase	desplazamiento	numArmas
Kongo	32000	8
North Carolina	37000	9

```
SELECT c.clase, c.desplazamiento, c.numArmas  
FROM Barcos b, Clases c  
WHERE b.clase=c.clase  
AND b.nombre = ANY (SELECT barco FROM RESULTADOS WHERE batalla="Guadalcanal");
```

- c) Clases de barco para los que existe un barco miembro de esa clase.

clase	numero
Colorado	1
Congo	4
Iowa	4
North Carolina	2
Renown	2
Revenge	5
Suffolk	1
Tennessee	2
Yamato	2

```
SELECT c.clase, COUNT(*) numero
FROM Barcos b, Clases c
WHERE b.clase=c.clase
GROUP BY c.clase
HAVING COUNT(*)>=1;
```

- d) Barcos que resultaron dañados en una batalla, pero sobrevivieron y pelearon en otra batalla posterior.

barco
Kirishima

```
SELECT r.barco
FROM Resultados r, Batallas bat
WHERE r.batalla=bat.nombre
and r.resultado= "dañado"
and r.barco in (select barco
from resultados r2, batallas bat2
where r2.batalla=bat2.nombre
and r2.barco=r.barco and bat.fecha < bat2.fecha);
```

- e) Barcos con desplazamiento superior a 35.000 toneladas.

nombre	clase	desplazamiento
Iowa	Iowa	46000
Missouri	Iowa	46000
New Jersey	Iowa	46000
Wisconsin	Iowa	46000
North Carolina	North Carolina	37000
Washington	North Carolina	37000
Musashi	Yamato	65000
Yamato	Yamato	65000

```
SELECT b.nombre, b.clase, c.desplazamiento
FROM clases c, barcos b
WHERE b.clase=c.clase AND c.desplazamiento>35000;
```

- f) Batallas en las que participaron al menos tres barcos del mismo país.

batalla	pais	numero
Surigao Strait	USA	3

```
SELECT r.batalla, c.pais, COUNT(*) numero
FROM resultados r, barcos b, clases c
WHERE r.barco=b.nombre AND b.clase=c.clase
GROUP BY r.batalla, c.pais
HAVING COUNT(*)>2;
```

- g) Batallas en las que participaron barcos de la clase Congo.

batalla	nombre	clase
Guadalcanal	Kirishima	Congo
North Cape	Kirishima	Congo

```
SELECT r.batalla, b.nombre, b.clase
FROM resultados r, barcos b
WHERE b.nombre=r.barco AND b.clase="Congo";
```

- h) Nombre de las clases de barcos tales que al menos un barco de la clase se hundió en una batalla.

clase
Congo
North Carolina
Yamato

```
SELECT c.clase
FROM resultados r, barcos b, clases c
WHERE r.barco=b.nombre AND b.clase=c.clase AND r.resultado="hundido"
GROUP BY c.clase;
```

- i) Número de clases distintas de barcos.

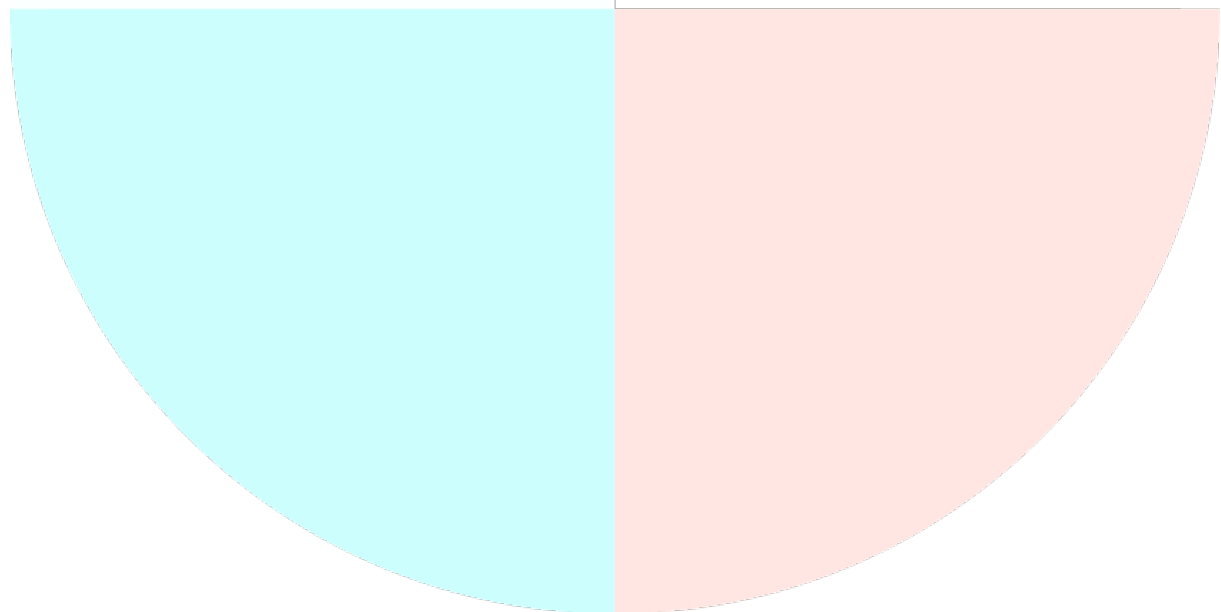
numero
9

```
SELECT COUNT(DISTINCT clase) numero FROM barcos;
```

j) Para cada clase, encontrar el año en el que fue botado el barco más antiguo de la clase.

clase	nombre	botado
Congo	Congo	1913
Iowa	Iowa	1941
North Carolina	North Carolina	1941
Renown	Renown	1913
Revenge	Revenge	1915
Suffolk	Suffolk	1924
Tennessee	Tennessee	1920
North Carolina	Washington	1941
Colorado	West Virginia	1920
Yamato	Yamato	1941

```
SELECT b.clase, b.nombre, b.botado
FROM barcos b
WHERE b.botado <= ALL (SELECT botado FROM barcos WHERE clase=b.clase);
```



Enunciado 6

Responda a las siguientes cuestiones:

- a) Dadas las tablas A y B, calcular el resultado de A **UNION** B y A **UNION ALL** B.

Tabla A
10
10
20
20
50

Tabla B
0
10
10
20

A union B	A union all B
0	10
10	10
20	20
50	20
	50
	0
	10
	10
	20

- b) Dadas las tablas anteriores A y B, calcular el resultado de A **INTERSECT** B y A **INTERSECT ALL** B.

A intersect B	A intersect all B
10	10
20	10
	20

- c) Dadas las tablas anteriores A y B, calcular el resultado de A **EXCEPT** B y A **EXCEPT ALL** B.

A minus B	A minus all B
50	20
	50